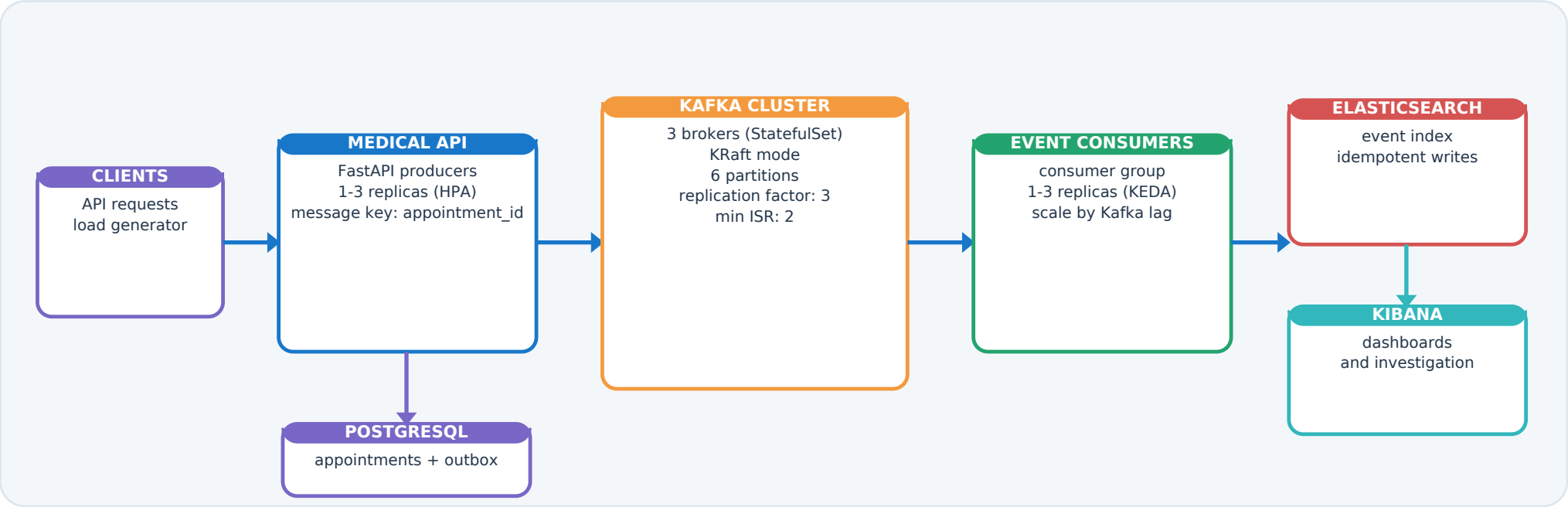


Target Architecture

Scalable FastAPI producers, a resilient three-broker Kafka cluster, independently scaled consumers, persistent storage, search analytics, and GitOps-controlled delivery.



Producer layer	Medical API scales from 1 to 3 replicas. Every replica can publish appointment events.
Kafka layer	Three fixed brokers provide replication and broker-failure tolerance. Brokers are not HPA-scaled.
Consumer layer	One to three workers share six partitions. KEDA scales workers using consumer lag.
Data layer	PostgreSQL is authoritative. Elasticsearch is a searchable event projection visualized through Kibana.

Scale, Reliability and Event Design

Medical API producers	1 replica	3 replicas	HPA: min 1, max 3
Kafka brokers	3 brokers	3 brokers	Fixed StatefulSet; planned/manual resizing
Topic partitions	6	6	At least 3 partitions for 3 active consumers
Topic replication	RF=3, min ISR=2	RF=3, min ISR=2	Survive loss of one broker
Consumers	1 replica	3 replicas	KEDA: min 1, max 3; scale by lag
Elasticsearch / Kibana	1 / 1	1 / 1	Single-node local learning setup

Event contract

Each event carries a unique event_id, event type, appointment ID, schema version, timestamp and trace ID. The Kafka key is appointment_id, preserving order for one appointment within a partition.

Example event

```
{
  "event_id": "uuid",
  "event_type": "appointment.created",
  "appointment_id": 123,
  "schema_version": 1,
  "occurred_at": "..."
```

Reliability controls

Producer: acks=all, idempotence enabled, retries, unique transactional ID when transactions are used.

Database: appointment and outbox event written in one PostgreSQL transaction.

Consumer: manual offset commits, retry policy, dead-letter topic, idempotency using event_id.

Observability: consumer lag, failed events, retries, under-replicated partitions, processing latency and Pod restarts.

Local capacity note: the full 3+3+3 demonstration is temporary on a 16 GB workstation. Normal learning mode uses 1 producer, 3 brokers and 1 consumer, with strict memory limits.

Kafka Delivery Semantics Laboratory

The project implements and deliberately tests each semantic. The normal application mode is at-least-once plus idempotency.

At-most-once	Commit offset before processing	No typical duplicates, but a crash can lose a message	Learning experiment; noncritical telemetry only
At-least-once	Process successfully, then commit offset	Avoids loss; crash before commit can redeliver	Default consumer mode + idempotent event_id
Exactly-once	Kafka input offset and Kafka output committed in one transaction	Atomic inside Kafka when producers/consumers are configured correctly	Kafka-to-Kafka transaction laboratory
Effectively-once	Outbox + unique event_id + idempotent database/search writes	Retries may occur, but business result is written once	PostgreSQL and Elasticsearch integration

Failure tests and acceptance criteria

Test	Expected observation
Kill consumer before processing	At-most-once can lose the event because its offset was already committed.
Kill consumer after write, before commit	At-least-once redelivers; idempotency prevents a duplicate business result.
Stop one Kafka broker	Producers and consumers continue while replicas remain in sync.
Replay the same event_id	PostgreSQL uniqueness and Elasticsearch document IDs prevent duplicate results.
Create large consumer lag	KEDA scales consumers from 1 toward 3 and partitions rebalance.

Important: Kafka's strict exactly-once guarantee applies to Kafka transactions. A single atomic transaction cannot automatically include Kafka, PostgreSQL and Elasticsearch; those integrations use an outbox and idempotent consumers to achieve effectively-once business results.

Phased Implementation Plan

1	Persistent foundation	Add PostgreSQL, migrations, appointments table, outbox_events and processed_events.
2	Kafka platform	Deploy 3 KRaft brokers; create 6-partition topic with RF=3 and min ISR=2.
3	Producer	Publish appointment events from 1-3 FastAPI replicas; use keys, schemas, acks=all and idempotence.
4	Consumers	Build Python indexer/notification workers; manual commits, retries, dead-letter topic and idempotency.
5	Search analytics	Index events in Elasticsearch and build Kibana dashboards for volume, errors and latency.
6	Semantics laboratory	Run at-most-once, at-least-once, exactly-once Kafka and effectively-once sink experiments.
7	Scaling and resilience	HPA producers 1-3; KEDA consumers 1-3; kill broker/Pods and verify recovery.
8	CI and registry	GitHub Actions: pytest, integration tests, Helm validation, image build/scan and GHCR push.
9	GitOps delivery	Argo CD updates the desired state; Argo Rollouts creates preview and supports controlled promotion.
10	Observability	Datadog or Prometheus dashboards and alerts for lag, errors, ISR, latency and restarts.
11	Security and hardening	Secrets management, RBAC/AppProject, network policies, resource limits and retention.
12	Portfolio evidence	README, architecture diagram, runbook, failure-test results and interview demonstration script.

Definition of done

Milestone A	Appointment is persisted and its event reliably reaches Kafka.
Milestone B	Three brokers tolerate one broker failure; producer and consumers scale to three.
Milestone C	Kibana displays searchable events without duplicate business results.
Milestone D	Git push triggers CI, registry publication, Argo CD sync, preview test and promotion.